

Boosting LLM Latent Reasoning with Predicted Latent Reasoning Steps

Xiayang Jin¹, Xinzhe Juan¹, Weiyi Wang¹, Haoran Zhang¹

¹University of Michigan
{ausummer, juanzx, wweiyi, zhhaoran}@umich.edu
Group 14

Abstract

Large language models (LLMs) typically improve reasoning via explicit Chain-of-Thought (CoT) prompting, but this incurs high token and compute overhead because every intermediate step is generated as text. The Chain of Continuous Thought (COCONUT) framework replaces these textual traces with recurrent “latent thoughts” in hidden-state space, claiming to preserve or improve accuracy while drastically reducing token usage. In this work we replicate COCONUT on GPT-2 (124M) across GSM8k, ProntoQA, and ProsQA, and probe its architectural limits by scaling down to a 14M-parameter Pythia model. On GPT-2 we largely confirm the original findings: on ProntoQA, COCONUT slightly improves accuracy while cutting token usage by about 75%, and on ProsQA it substantially outperforms CoT with similar savings. On GSM8k, both methods underperform previously reported numbers, likely due to reduced training epochs, but COCONUT still attains modest accuracy gains with roughly 30% fewer tokens. Experiments with Pythia-14m reveal a sharp capacity dependence: latent reasoning remains beneficial on ProsQA, but degrades ProntoQA performance and completely fails on GSM8k despite smoothly decreasing training loss.

Motivated by the observation that different questions prefer different latent depths, we propose an adaptive latent-length controller for COCONUT. We estimate a per-question distribution over latent steps (0–6) from the question’s final hidden state using a lightweight three-layer MLP trained on GSM8k, and apply a test-time logit bias to trade off accuracy against average latent length. An oracle analysis shows that perfect step selection could raise GSM8k accuracy from 70.3% for the best fixed step to 80.5% under the same step budget. Our learned predictor does not reach the oracle but consistently surpasses all fixed-step baselines at matched budgets while enabling smooth, ahead-of-time control of latent compute.

Introduction

The rapid evolution of Large Language Models (LLMs) has been driven by the Transformer architecture’s ability to model complex probability distributions over sequences of discrete tokens. While these models excel at pattern matching and syntactic structure, they encounter a significant “reasoning wall” when tasked with multi-step logical deduction or strategic planning. The dominant methodology for overcoming this barrier has been explicit Chain-of-Thought

(CoT) prompting, which forces the model to externalize intermediate reasoning steps as natural language text. However, this reliance on explicit token generation introduces a “Reasoning Bottleneck”: every intermediate step incurs the computational cost of a full forward pass and forces high-dimensional internal concepts to collapse into discrete, often suboptimal, vocabulary tokens.

Research Problem and Significance. Our project focuses on the paradigm of **Latent Reasoning**, specifically identifying the architectural lower bounds of the Chain of Continuous Thought (COCONUT) framework. The key research problem we address is: *Can the capabilities of latent reasoning be successfully replicated and transferred to smaller parameter-space models, such as Pythia-14m, or does this capability require the massive dimensionality of larger models?* This question is critical for democratizing AI; if latent reasoning is efficient and scale-invariant, it could enable powerful reasoning on consumer-grade hardware. Conversely, if it relies on a capacity threshold, its utility may be restricted to industrial clusters.

Core Paper Results. The core paper by Hao et al. (2024) claims that COCONUT achieves comparable or superior accuracy to CoT while drastically reducing token consumption. Specifically, they report that on the planning-intensive **ProsQA** dataset, COCONUT outperformed CoT (97.0% vs 77.5%) while using significantly fewer tokens. We have focused our replication efforts on validating these efficiency gains and testing the “breadth-first” search hypothesis on the Pythia-14m architecture.

Proposed Extension. Our proposed extension introduces a **latent reasoning step predictor**. We proposed that a fixed number of latent steps as used in Hao et al. (2025) is inefficient. By training a lightweight controller to predict the optimal reasoning depth c for each query, we aim to dynamically allocate compute resources—allowing the model to “think” longer for hard problems and respond instantly to simple ones. Our code is available at <https://github.com/EECS498-Team14/coconut>.

Related Works

Recent work in LLM reasoning has focused on structuring the generation process. **Chain-of-Thought (CoT)** (Wei

et al. 2022) demonstrated that intermediate text steps improve performance but at high computational cost. **Tree-of-Thoughts (ToT)** (Yao et al. 2023) and **Stream-of-Search** (Gandhi et al. 2024) extended this to explicit search algorithms, allowing models to explore multiple reasoning paths. However, these methods operate in the discrete token space.

Closest to our work is **COCONUT** (Hao et al. 2025), which moves this search into the continuous latent space. Other recent efforts like **CODI** (Shen et al. 2025) and **Soft-CoT** (Xu et al. 2025) also explore compressing reasoning, but often rely on distillation from larger teacher models. Our work distinguishes itself by stress-testing the architecture on significantly smaller models (14M parameters) and proposing a dynamic length controller to bridge the gap between efficiency and flexibility.

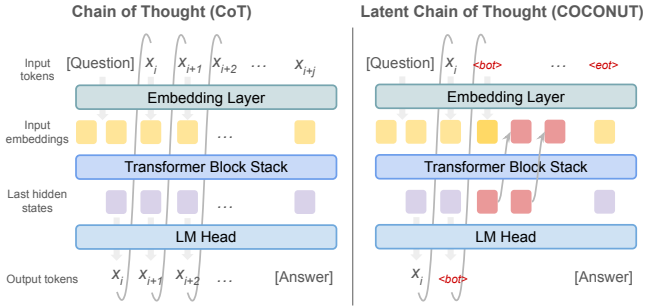


Figure 1: Comparison between standard Chain-of-Thought (CoT) and Latent CoT (COCONUT). **Left:** CoT generates explicit reasoning steps as language tokens and feeds them back as inputs. **Right:** COCONUT replaces these intermediate tokens with continuous latent thoughts, using hidden states as the next-step inputs.

Background

This section establishes a robust theoretical framework covering the evolution of reasoning in Large Language Models, the specific mechanics of the COCONUT architecture, and the datasets used for evaluation.

From Next-Token Prediction to Explicit Reasoning

Standard LMs lack depth for multi-step reasoning, and while CoT adds explicit intermediate steps to extend computation, it is costly and compresses rich internal states into discrete tokens.

Standard Language Model Training. Large language models (LLMs) are typically trained as next-token predictors: given a context $x = (x_1, \dots, x_t)$, the model estimates $P(x_{t+1} | x)$ and learns to capture statistical and syntactic regularities. This setup works well for many tasks but struggles when problems require long, serial chains of computation.

Computation Depth Limitation. In standard *direct answer* settings, the model must map a problem x directly to a solution y in a single forward pass. For multi-digit arithmetic or multi-step logical reasoning, the effective computational depth required often exceeds what a fixed-depth

Transformer can implement. The model simply cannot perform enough serial computation before producing an answer.

Chain-of-Thought (CoT) Paradigm. Chain-of-Thought (CoT) prompting (Wei et al. 2022) addresses this limitation by expanding generation. Instead of emitting y directly, the model first produces intermediate reasoning steps $z = (z_1, \dots, z_k)$ and then answers based on both x and z . Each additional step z_i corresponds to another forward pass, effectively allocating more compute and using the generated text as an explicit scratchpad that later tokens can attend to.

However, CoT constrains every reasoning step z_i to be a discrete vocabulary token. At each step, the model must collapse its high-dimensional internal state into a specific word or symbol, potentially discarding nuanced information. Token-by-token generation also increases inference cost, as every reasoning step requires running the full output projection over the entire vocabulary.

The Chain of Continuous Thought (COCONUT)

The COCONUT framework (Hao et al. 2025) proposes a paradigm shift from discrete to continuous reasoning, leveraging the insight that hidden states, the vector outputs from the final Transformer layer before the language head, contain rich semantic information that need not be immediately collapsed to discrete tokens, as is shown in Figure 1.

Continuous Latent Thoughts Mechanism. Instead of requiring vocabulary token outputs at every reasoning step, COCONUT relaxes this constraint for intermediate computations:

1. **Input Processing.** The model processes the input question x and any preceding context.
2. **Latent Recurrence.** Rather than passing the final hidden state h_t through the language head (LM head) to sample a token, h_t is fed directly back into the model as the input embedding for time step $t + 1$. Normally, inputs are discrete token indices mapped to static embeddings $E(x_{t+1})$; in COCONUT, the input becomes the dynamic vector h_t .
3. **Thought Loop.** This recurrence continues for a predetermined number of steps c (the *latent length*). During this phase, the model operates entirely in continuous embedding space, manipulating high-dimensional vectors without discrete vocabulary interaction.
4. **Decoding.** After c latent steps, the model switches back to standard generation to produce the final answer y .

This mechanism theoretically enables optimization in latent space, exploring reasoning trajectories that may lack direct textual representations. It also eliminates the computational overhead of output projection (where vocabulary size $|V|$ makes the final matrix multiplication expensive) for all intermediate steps. A detailed example is shown in Figure 2.

Curriculum Learning Strategy. Training models to utilize continuous reasoning space requires careful design. COCONUT employs a multi-stage curriculum that gradually

[Question]:

Kylar went to the store to buy glasses for his new apartment. One glass costs \$5, but every second glass costs only 60% of the price. Kylar wants to buy 16 glasses. How much does he need to pay?



Language CoT

[Step 1]: 16 glasses: every second glass is 60% of \$5 $\rightarrow 0.6 \cdot 5 = \3 .

[Step 2]: So each pair costs $5 + 3 = \$8$.

[Step 3]: 16 glasses = 8 pairs.

[Step 4]: 8 pairs $\times \$8 = \64 .

[Answer]: the answer is 64

Latent CoT

<bot>

<Latent Thought 1>

<Latent Thought 2>

<Latent Thought 3>

<Latent Thought 4>

<eot>

[Answer]: 64

Latent Space

First, I need the cost of every second glass

Then, I can calculate cost of each pair

After that, I need the cost of each pair

Finally, I can calculate the cost of 8 pairs

Figure 2: Comparison of Language CoT and Latent CoT. Language CoT generates explicit textual reasoning steps, while Latent CoT performs reasoning entirely through continuous latent states between <bot> and <eot>, emitting only the final answer. The example shows that both approaches reach the same solution but differ in how intermediate reasoning is represented.

transitions from explicit text-based reasoning to fully latent computation as displayed in Figure 5:

Stage 0 (Language CoT). The model is fine-tuned on datasets with fully explicit reasoning steps in text, initializing it with the logical patterns needed for the task (e.g., step-by-step math problem solving).

Stage 1 (Partial Latent). Early reasoning steps are “masked”: their text is removed and replaced with latent thought tokens, denoted by special control tokens <bot> (Beginning of Thought) and <eot> (End of Thought). The model learns to predict remaining explicit steps based on these latent inputs.

Progressive Masking. As training advances, increasingly more text steps are converted to latent thoughts, forcing the model to compress longer reasoning chains into continuous states.

Final Stage (Fully Latent). All intermediate text is removed. The model must transition directly from question to answer, bridging the entire logical gap via continuous thoughts alone.

This progressive curriculum is hypothesized to help the model internalize reasoning processes, effectively “distilling” explicit CoT into compact latent representations.

Experimental Datasets and Metrics

Our evaluation employs three primary benchmarks, each probing distinct dimensions of reasoning capability.

- **GSM8k (Grade School Math).** GSM8k (Cobbe et al. 2021) consists of multi-step arithmetic word problems requiring several sequential operations.
- **ProntoQA (Synthetic Logic).** ProntoQA (Saparov and He 2022) generates synthetic first-order logic puzzles with explicit rule chains and queries.
- **ProsQA (Planning and Search).** ProsQA (Hao et al. 2025) focuses on planning-style proof tasks, where reaching the correct conclusion may require exploring multiple candidate paths.

Performance Metrics. Beyond task-specific accuracy, we track token count (reflecting computational cost and latency). These metrics are exactly the ones used in the COCONUT paper (Hao et al. 2025).

Methodology, Results, and Discussion

This section presents our experimental setup, results, and analysis. We first describe our methodology, including model selection, infrastructure adaptations, and key implementation modifications. We then report replication results on GPT-2 and new experiments on Pythia-14m. Finally, we discuss how our findings align with or diverge from the original paper.

Methodology

Model Selection. While the original COCONUT paper only conducts experiments on GPT-2 (Radford et al. 2019), we selected EleutherAI’s Pythia-14m (Biderman et al. 2023) to test the lower bound of latent reasoning capabilities. We also conducted experiments with GPT-2 to match the original paper.

Infrastructure Adaptations. The original setup used $4 \times$ NVIDIA A100 (80 GB) GPUs with Fully Sharded Data Parallel (FSDP) and a batch size of 32. Our single A100 (40 GB) GPU required reducing the local batch size to 16 while implementing an 8-step gradient-accumulation schedule, yielding an effective batch size of 128. This smoothed the gradient landscape sufficiently for stable convergence.

Key Implementation Modifications. We made three critical modifications to enable cross-architecture replication and comprehensive evaluation. First, we refactored the codebase to use HuggingFace’s unified `AutoModel` interface, enabling seamless switching between GPT-2 and Pythia without model-specific code changes. Second, we instrumented the evaluation pipeline to track the average token generation counts, which is absent in the codebase. Third, we initialized special control tokens (<bot>, <eot>) using semantically relevant embeddings rather than random noise, which significantly stabilized early-stage training. We followed the original staged curriculum but found that Pythia-14m required more gradual text-to-latent transition rates to prevent training collapse.

Replication Results

GPT-2 (124M) Results. Our GPT-2 experiments largely validated the original paper’s core claims while revealing

some performance gaps attributable to differences in training duration.

Table 1: GPT-2 results across three benchmarks. "Paper" columns report numbers from the COCONUT paper. ProntoQA and ProsQA run 25 epochs (50 epochs in original paper); GSM8k CoT runs 1 epoch and COCONUT runs 3 epochs. (25 epochs in original paper)*

Dataset	Method	Metric	Ours	Paper
ProntoQA	CoT	Accuracy	98.1%	98.8%
		Avg tokens	93.59	92.5
	COCONUT	Accuracy	99.9%	99.8%
		Avg tokens	23.88	9.0
ProsQA	CoT	Accuracy	79.4%	77.5%
		Avg tokens	52.03	49.4
	COCONUT	Accuracy	94.4%	97.0%
		Avg tokens	—	14.2
GSM8k	CoT	Accuracy	20.1%	42.9%
		Avg tokens	32.82	25.0
	COCONUT	Accuracy	23.0%	34.1%
		Avg tokens	22.81	8.2

*Token counts exceed reported values due to incomplete curriculum progression. We trained fewer epochs due to computational constraints and the increasing time cost per epoch as training progresses (more latent thoughts require more sequential forward passes).

On ProntoQA and ProsQA, we successfully replicated the original findings: COCONUT achieves comparable or superior accuracy while dramatically reducing token count. For ProntoQA, COCONUT slightly exceeded the paper’s reported accuracy while reducing tokens by roughly 75%. For ProsQA, COCONUT substantially outperformed CoT, confirming the planning-task advantage, though falling slightly short of the paper’s 97.0%.

On GSM8k, both methods underperformed relative to the paper, likely due to our shortened training schedule (1–3 epochs vs. the paper’s longer regimen). Nevertheless, COCONUT maintained its efficiency advantage with a 30% token reduction while still slightly improving over CoT accuracy.

Pythia-14m (14M) Results. Experiments with the substantially smaller Pythia-14m model revealed stark capacity-dependent limitations.

Table 2: Pythia-14m results across three benchmarks. ProsQA, ProntoQA runs 25 epoches, while the GSM8k runs 10 epoches.

Dataset	Method	Accuracy	Avg Tokens
ProntoQA	CoT	56.3%	96.91
	COCONUT	50.4%	23.81
ProsQA	CoT	52.8%	51.40
	COCONUT	63.6%	8.23
GSM8k	CoT	0.4%	26.51
	COCONUT	0.0%	12.20

ProsQA remained a success story even at the 14M scale:

COCONUT achieved a +10.8 percentage-point gain over CoT while reducing tokens by 84%. This confirms that planning tasks benefit from latent reasoning even in severely capacity-constrained models.

ProntoQA, however, showed a performance regression: accuracy dropped from 56.3% to 50.4% (approaching random guessing on this binary task), although token efficiency improved by 75%. We hypothesize that the small model’s 256-dimensional latent space cannot maintain crisp logical truth values through extended reasoning chains.

GSM8k exhibited complete failure: 0.0% accuracy versus CoT’s already-poor 0.4%. Notably, training loss decreased normally throughout the curriculum stages, indicating that the model minimized prediction error on training data. This counterintuitive phenomenon, namely decreasing training loss with near-zero evaluation accuracy, suggests severe underfitting without learning generalizable arithmetic operators.

Discussion

Alignment with Original Paper. Our GPT-2 results confirm both core claims of the original COCONUT paper: (1) a 75%–84% token reduction across all benchmarks, and (2) superior performance on ProsQA for planning-intensive tasks (Hao et al. 2025).

Sources of Disagreement. Performance divergence arises from three main factors:

- **Model Capacity.** The most critical factor is embedding dimensionality. GPT-2’s hidden size provides sufficient space to maintain distinct encodings for different reasoning states, while Pythia-14m’s 256 dimensions suffer semantic collision, where disparate states map to overlapping vectors and prevent precise decoding. This explains success on fuzzy planning tasks (which tolerate approximation) but failure on arithmetic (which demands exact values).
- **Task Precision Requirements.** Performance follows a precision hierarchy: planning problems benefit from maintaining superpositions of paths (a natural affordance of continuous representations), logical problems require crisp but simple Boolean states (challenging but feasible), and arithmetic demands exact symbolic manipulation where numbers like “45” and “46” must be categorically distinct despite semantic similarity.

Extensions

This section introduces our proposed extension of an adaptive latent-length controller for COCONUT. We first motivate this extension by showing that different questions benefit from different numbers of latent reasoning steps. We then analyze the feasibility of adaptive allocation through oracle experiments. Finally, we describe our predictor architecture and evaluate its performance on GSM8k.

Motivation and Setup

During our replication, we notice an interesting phenomenon: different questions are better solved with different

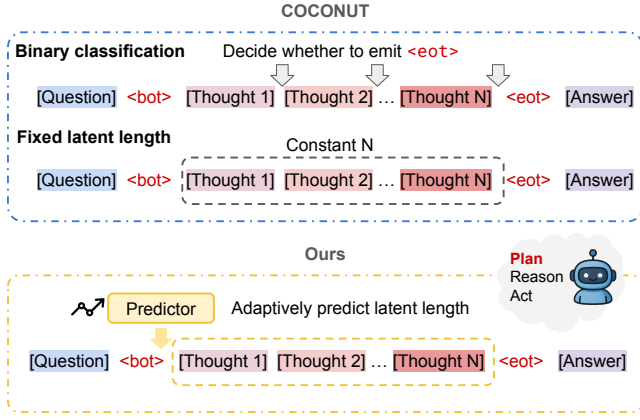


Figure 3: Comparison between COCONUT and our adaptive latent-length approach. COCONUT either predicts an `<eot>` boundary via binary classification or uses a fixed latent length N , producing a constant number of latent thoughts for all questions. Our method introduces a lightweight predictor that adaptively selects the latent reasoning length based on the input question, enabling flexible allocation of latent compute while maintaining the same latent reasoning formulation.

latent reasoning steps, indicating that the fixed latent reasoning step approach utilized in COCONUT (Hao et al. 2025). In this extension, we want to explore: *With (approximately) the same total latent reasoning budget¹, can we boost LM accuracy by cleverly allocating different budget to different questions?* In the context of COCONUT, our goal is to construct an informed latent reasoning step count predictor mapping each question to its allocated latent reasoning step. We also include a detailed discussion of the intuition behind this design in the Appendix.

Checkpoint for GSM8k. We have a discussion of the checkpoint we used for GSM8k in this section in the Appendix due to page limitations.

Analysis of Feasibility

In this section, we start by quantitatively justifying our finding that different questions are better solved with different latent reasoning steps. Then we prove empirically why it is feasible to boost accuracy with a predictor.

Settings. We collect a dataset of questions and the allocated latent reasoning steps that result in correct answer for each question. We randomly sample 100,000 questions from GSM8k train set and run the pretrained GPT-2.

Distribution of Correct Steps. We consider possible latent reasoning steps from $\{0, 1, 2, \dots, 6\}$, as in COCONUT (Hao et al. 2025). From the dataset we collected, for all possible steps, we compute the ratio of questions that could be correctly answered by GPT-2 when allocated that number of latent reasoning steps but not when allocated

¹We justify our choice of investigating latent reasoning steps rather than total generated token counts in the Appendix.

steps smaller than that number. The results are summarized in Table 3 in the Appendix. We conclude that the smallest number of steps needed to answer each question correctly is diversified (with a standard deviation of 1.65), and that any fixed number of steps is not ideal.

Oracle Performance. We also measured the performance of an “oracle”, which always selects a step count that makes it answers correctly to a specific question (if possible). We found that the oracle accuracy on the train set is 80.47%, significantly higher than 70.31% the best accuracy among all fixed step choices (which is 6 steps in this case). This indicates that there is clear room for improvement over fixed-step approaches when the predictor is accurate enough.

Methods

In this section, we describe the detailed methods we apply to construct the predictor. We note that we also tried alternative plausible approaches, which generally perform worse than our current approach. Please check the Appendix for detail.

Input and Output. While the direct form of the predictor would map each question to a predicted step count, we use a slightly different formulation of predictor input and output for practical considerations. **Input:** We use the *hidden state* right after the language model finishes reading the question texts. This is helpful because we don’t need to encode the question again during inference, and can simply utilize the encoded state which must be generated anyway in COCONUT. **Output:** Instead of learning a single value as the predicted step, the model should output a distribution over all steps in $\{0, 1, 2, \dots, 6\}$. This is because each question could correspond to multiple correct steps. Formally, we are learning a model $f(\cdot; \theta)$ parameterized with θ with maps from $\mathbb{R}^H$ to $\mathbb{R}^7$ where H is the hidden state dimension and $\mathbb{R}^7$ corresponds to the space of the logits of $\{0, 1, 2, \dots, 6\}$.

Model Architecture and Loss. We use a three-layer Multilayer Perceptron (MLP). Concretely, the hidden state input is first mapped to dimension 1024, then mapped to dimension 512, and finally mapped to a distribution over $\{0, 1, 2, \dots, 6\}$. We use the cross-entropy loss.

Class Weights. We find that the target class proportion in our multi-target dataset is imbalanced², which motivates us to weight each step differently. Concretely, the incorrect penalty in cross-entropy loss for each step is weighted by the inverse of the frequency of that step in the train set.

Logit Bias. As we cannot directly control the average latent reasoning step outputted by the predictor, we decide to add a continuous parameter that we can tune at *test-time* so that we can use the same model throughout. Concretely, we bias the logit corresponding to step s with $\kappa \cdot s$, where $\kappa \in \mathbb{R}$ controls the resulted average step count: Positive κ gives bigger average token count. During inference, we greedily select the step s with the highest biased logit:

$$s = \arg \max_{s' \in \{0, 1, \dots, 6\}} \{f(h; \theta)_{s'} + \kappa \cdot s'\}$$

²Please check the Appendix for more detail.

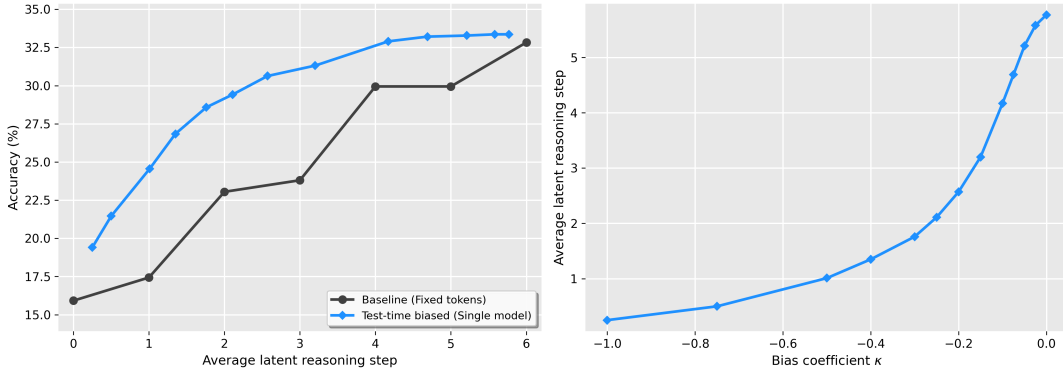


Figure 4: Test-time logit biasing enables accuracy improvements across different latent reasoning budgets. **(Left)** A single predictor model with adjustable bias coefficient κ (blue) boosts accuracy, exceeding baseline vanilla COCONUT with fixed reasoning steps (gray). Unlike baseline approaches requiring separate strategies for each budget level, the biased predictor adapts seamlessly to any desired reasoning budget. **(Right)** The bias coefficient κ provides smooth, monotonic, and high-coverage control over average latent reasoning steps, enabling fine-grained tuning of reasoning depth at inference time without retraining the predictor.

where $f(\cdot; \theta)$ is the evaluated model and h is the hidden state corresponding to the question.

Experiments and Results

Due to computational limitations and practical considerations³, we only examined our proposed method on GSM8k. However, as GSM8k is the most challenging one among all three datasets for COCONUT, it still indicates the potential of our extension.

Discussion We conclude that our method have the following advantages over the vanilla COCONUT (and its binary classification variant as in Figure 1):

1. Our predictor results in **better task accuracy** than the vanilla COCONUT under our setting across different budget of latent reasoning steps, as in Figure 4 the predictor curve is above the COCONUT fixed-step baseline.
2. Our predictor enables **continuous and high-coverage control** over latent reasoning budget (by tuning κ), which might be beneficial for scenarios where fine-grained budget management is necessary.
3. Our predictor is **ahead-of-time**, which means it can be applied before the actual reasoning begins. By contrast, the binary classification variant must be applied during the reasoning procedure. Ours is more preferable under multiple circumstances. For example, one can pre-allocate computational resources or filter the dataset based on predictor outputs without costly reasoning.
4. The cost of our predictor is **independent of reasoning lengths**, while the binary classification COCONUT produces additional cost per reasoning step. This feature is convenient for tasks that require deep reasoning.

³The COCONUT performance on ProntoQA and ProsQA is near perfect, as shown in Table 1. Further, the worst performing step on ProsQA gives an accuracy of 93.4%. Therefore, these datasets are already easy enough to COCONUT, and there is little room for further improvement.

Conclusions

In this work, we successfully replicated the COCONUT framework on GPT-2 (124M), validating its ability to outperform explicit Chain-of-Thought (CoT) on planning tasks like ProsQA while reducing token consumption by approximately 75%. However, our experiments scaling down to Pythia-14m revealed a critical “capacity dependence” in latent reasoning. While the small model’s low-dimensional latent space remained effective for fuzzy planning, it suffered from semantic collision that prevented the precise arithmetic and logical operations required for GSM8k and ProntoQA.

As an extension, we proposed a predictor that predicts latent token steps before latent reasoning.. By dynamically predicting the optimal latent depth per query, our method consistently outperformed fixed-step baselines on GSM8k, demonstrating that dynamic compute allocation can effectively balance accuracy and cost. Future work could focus on training a more accurate latent-length controller and extending the adaptive mechanism to a broader range of reasoning tasks.

Societal Impact

By demonstrating that latent reasoning can drastically reduce token-generation cost, this approach highlights a promising new paradigm for efficient and scalable reasoning. However, the “black box” nature of continuous latent thoughts removes the human-readable audit trail provided by explicit CoT, potentially complicating interpretability and safety monitoring.

References

- Biderman, S.; Schoelkopf, H.; Anthony, Q.; Bradley, H.; O’Brien, K.; Hallahan, E.; Khan, M. A.; Purohit, S.; Prashanth, U. S.; Raff, E.; Skowron, A.; Sutawika, L.; and van der Wal, O. 2023. Pythia: A Suite for Analyzing Large Language Models Across Training and Scaling. arXiv:2304.01373.

Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, J.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. In *Advances in Neural Information Processing Systems*, volume 34, 21191–21203.

Gandhi, K.; Lee, D. H. J.; Grand, G.; Liu, M.; Cheng, W.; Sharma, A.; and Goodman, N. D. 2024. Stream of Search (SoS): Learning to Search in Language. ArXiv preprint, arXiv:2404.03683.

Hao, S.; Sukhbaatar, S.; Su, D.; Li, X.; Hu, Z.; Weston, J.; and Tian, Y. 2025. Training Large Language Models to Reason in a Continuous Latent Space. In *Proceedings of the 2nd Conference on Language Modeling*. COLM 2025.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770–778.

Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I.; et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8): 9.

Saparov, A.; and He, H. 2022. A Probabilistic Logic Programming Approach to Question Answering with Uncertain Knowledge. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*.

Shen, Z.; Yan, H.; Zhang, L.; Hu, Z.; Du, Y.; and He, Y. 2025. CODI: Compressing Chain-of-Thought into Continuous Space via Self-Distillation. arXiv:2502.21074.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention Is All You Need. arXiv:1706.03762.

Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E. H.; Le, Q. V.; and Zhou, D. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Advances in Neural Information Processing Systems*. NeurIPS 2022.

Xu, Y.; Guo, X.; Zeng, Z.; and Miao, C. 2025. SoftCoT: Soft Chain-of-Thought for Efficient Reasoning with LLMs. arXiv:2502.12134.

Yao, S.; Yu, D.; Zhao, J.; Shafraan, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. In *Advances in Neural Information Processing Systems*. NeurIPS 2023.

Individual Contributions

- **Xiayang Jin.** Implemented data loaders and training pipelines for GSM8k, ProntoQA, and ProsQA on GPT-2 and Pythia, ran most replication experiments, logged metrics and token counts, and drafted the Methodology and Results sections.
- **Xinzhe Juan** Designed the adaptive latent-length controller, including predictor architecture, class weighting, and logit-bias scheme, implemented controller training and evaluation on GSM8k, and wrote most of the Extensions, Discussion, and societal impact analysis.

- **Weiyi Wang.** Led literature review on latent reasoning and COCONUT, drafted Background and Related Work, helped design comparisons between CoT and COCONUT, created figures and tables, and edited for clarity and coherence.

- **Haoran Zhang.** Coordinated project scope and timeline, implemented code changes for control tokens and curriculum schedule, analyzed failure modes on Pythia-14m, wrote parts of Abstract, Introduction, Discussion, and finalized structure and formatting.

Appendix

Dataset

Here are some examples of what these datasets contain.

GSM8k (Grade School Math). GSM8k (Cobbe et al., 2021) is a standard benchmark for arithmetic reasoning. Problems are multi-step word problems requiring sequential operations. For example, the problem

“Janet has 5 apples. She buys 3 more, then gives half to her brother. How many does she have?”

requires computing $5 + 3 = 8$, then $8/2 = 4$. This dataset is particularly challenging for latent reasoning because arithmetic demands precision—“4” is correct while “4.1” is wrong—and encoding exact numerical values in continuous latent space is non-trivial compared to explicit discrete tokens. We measure performance using exact match accuracy on the numerical answer.

ProntoQA (Synthetic Logic). ProntoQA (Saparov and He, 2022) generates synthetic first-order logic puzzles with explicit rule chains. A typical problem provides rules like “Every wumpus is a feinschmecker” and “Sally is a wumpus,” then asks “Is Sally a feinschmecker?” This tests the model’s ability to maintain logical consistency over potentially deep implication chains. Unlike math, the reasoning involves boolean states rather than precise numerical values, but chains can extend to significant depths. Performance is measured by true/false classification accuracy.

ProsQA (Planning and Search). ProsQA (Hao et al., 2024) was introduced specifically to highlight latent reasoning strengths. It involves proof generation tasks where the path from premises to conclusions is non-linear and may require exploring multiple solution branches. The hypothesis is that CoT reasoning is “greedy”—it commits to the most likely next token at each step—while latent reasoning, being continuous, can maintain a superposition of possible paths (analogous to breadth-first search) before collapsing to a decision. We measure proof accuracy and expect COCONUT to significantly outperform CoT on this benchmark.

Justification of Latent Reasoning Steps

We have several reasons defending our choice of dealing with the number of latent reasoning steps instead of the total number of generated tokens. Technically, the current COCONUT pipeline does not offer any control over the number of tokens generated after leaving the latent reasoning phase. Additionally, during the replication we find that empirically,

while the latent reasoning phase is only responsible for a portion of the total cost, it is where the *true effective reasoning* takes place for COCONUT. Concretely, we measure the correlation between task accuracy and (1) latent reasoning step, or (2) the number of tokens generated after leaving latent reasoning. For both GSM8k and ProsQA, we observe that the correlation for (1) is generally positive (0.979 and 0.625) while the correlation for (2) is highly negative (-0.915 and -0.999), indicating that the model isn’t reasoning effectively in the post-latent phase. This implies that latent reasoning step is more intrinsic as a measure of reasoning budget, and we expect that the token count after latent reasoning could be reduced more easily than the latent reasoning step, although this topic is beyond the scope of our interest. Finally, it is not directly clear how to properly compare the cost of latent reasoning phase and the standard answer generation after leaving the latent reasoning as these are completely two different reasoning paradigms.

Checkpoint for GSM8k in Extension

As stated in the replication part, we only train the GPT-2 models on GSM8k for 1-3 epochs, far shorter than the paper’s choice, due to computational limitations. To minimize the impact on our extension experiments, we make use of an external set of public checkpoints replicated for the COCONUT paper⁴. The checkpoints are trained for more epochs. Further, to verify the validity of the checkpoints, we evaluate the performance of the checkpoints and observe that the checkpoints give close performance results to the ones in the COCONUT paper. For the following, we always use the last external checkpoint for consistency.

Distribution of Correct Steps

Table 3: Ratio of samples with each step count being the smallest “correct” step count in GSM8k.

Step	0	1	2	3	4	5	6
Ratio (%)	46.8	20.0	14.4	5.1	7.1	3.2	3.4

Alternative Approaches for Predictor

In this section, we discuss part of alternative approaches we tried for our extension and how and why they don’t work ideally. We fix $\kappa = 0$ for simplicity. Our adopted method produces a predictor with accuracy 33.36% in GSM8k under this setting.

Regression We tried another type training framework: conduct direct regression towards a correct step for each question. We tried three variants: (1) towards *minimum* correct step, (2) towards *maximum* correct step, (3) towards the *average* of all correct steps. The results are shown in Table 4. We conclude that none of these variants outperform our current method. We believe that a possible reason is the distribution of correct steps across samples could be different from a simple unimodal distribution. For example, it is

possible that step 0, 1, 5, and 6 are correct for a question, but not 2, 3, 4. In our case, regression cannot handle multiple possible targets properly.

Table 4: Accuracy with different regression-based predictor variants in GSM8k.

Type	Min.	Max.	Average
Accuracy (%)	21.00	32.68	30.02

Different Architectures We also tried more complex architectures beyond MLP. For example, we tried a ResNet (He et al. 2016) with hidden dimension 512 and block count 6, and also a Transformer (Vaswani et al. 2017) with 8 heads, 2 layers, and model dimension 64. However, they don’t meaningfully enhance the accuracy, yielding 33.13% and 33.21% accuracy on GSM8k each. We hypothesize that it could be because the structure of space of the hidden state, as the input to the predictor, is rather unclear, so these models that perform well on specific categories of spaces may not learn the features as well in our settings.

Hidden Dimensions For MLP, we also evaluated different choices of hidden dimensions. For example, we tried (2048, 512) and (1024, 1024) in place of (1024, 512). The resulted accuracies are 33.28% and 32.83% respectively, with no meaningful improvements from our current methods. We think that a possible reason is the (1024, 512) dimensions are already enough to effectively represent the features, so including more dimensions are not helpful.

Learning Rates We use learning rate 10^{-3} in our current method. We also tried different learning rates, but their resulted accuracies are lower than our current method. For example, the accuracy with learning rate 5×10^{-4} yields 32.83% accuracy.

Without Class Weights We tested without class weights. The resulted accuracy is 33.21%, a bit less than our current method. This is reasonable because class frequencies are not extremely imbalance, as in the next section.

Class Imbalance

As shown in Table 5, about 70% of the samples we collected include 6 as a correct step, while only 38% of the samples include 0. To mitigate the imbalance, we weight different steps differently.

Table 5: Ratio of samples with each step count being one of the “correct” step counts (if any) in GSM8k.

Step	0	1	2	3	4	5	6
Ratio (%)	38	41	54	55	64	66	70

⁴<https://huggingface.co/Esther22/coconut.Reproduction>

Illustration of Training Process

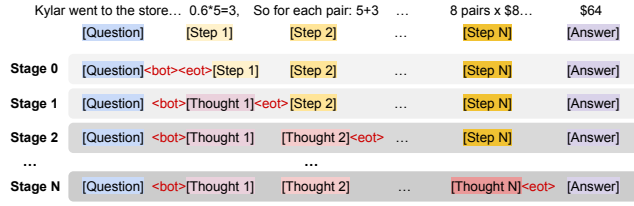


Figure 5: Progressive construction of latent Chain-of-Thought across training stages. Starting from language CoT supervision (Stage 0), each subsequent stage replaces an additional language reasoning step with a continuous latent thought between `<bot>` and `<eot>`. By Stage N, the entire reasoning process is represented in the latent space while only the final answer remains in natural language.

Intuition for Extension

Ideally, the predictor should allocate more latent reasoning budget to more *complex* questions, while limiting the budget for *straightforward* questions, by learning the *difficulty* for a given question, *before* actual reasoning takes place. This idea is inspired by the recent trend of integrating *planning mechanisms* into LLMs. For example, the Tree-of-Thoughts (ToT) framework (Yao et al. 2023), and Stream-of-Search (Gandhi et al. 2024) demonstrate that explicitly modeling a planning process enables LLMs to explore and evaluate multiple reasoning trajectories before committing to an answer. We hypothesize that a comparable internal planning signal can guide the latent reasoning depth of COCONUT.